

PBCLLM: Pareto Behavior Coevolution for LLM-Generated MOCO Heuristics

Contents

1	Core Idea	2
2	Evaluator and Archive	2
3	Quality Metrics	2
4	Pareto Behavior Trajectory (PBT)	3
5	Behavior Distance with Dynamic Time Warping (DTW)	3
6	Behavior Diversity Calculation	4
7	Population Selection and Evolution	4
8	Full Algorithm	5
8.1	Input and Initialization	5
8.2	Initial Population Construction	6
8.3	Evolution Loop	6
8.4	Selection Subroutine	7
8.5	Output	8
9	Summary	8

1 Core Idea

PBCLLM evolves LLM-generated heuristic programs for multi-objective combinatorial optimization (MOCO). Each individual is a heuristic h_θ that, given a Pareto archive A_t and instance data I , proposes a candidate solution x' . A heuristic is selected not only for final Pareto quality, but also for **distinct Pareto search behavior**.

The main selection objective is a two-objective maximization:

$$G(h) = (C_{HV}(h), D_{beh}(h)),$$

where $C_{HV}(h)$ is population-level hypervolume contribution and $D_{beh}(h)$ is behavior diversity based on executed search trajectories.

2 Evaluator and Archive

The evaluator assumes minimization objectives. For a maximization problem, objectives are negated. When heuristic h proposes a valid point y' , the archive updates as:

$$A_{t+1} = ND(A_t \cup \{y'\}),$$

where $ND(S)$ is the nondominated subset of S . The final front on instance j is the last archive: $P_{h,j} = ND(A_{h,j}^{\tau_L})$.

The evaluator also returns **archive trajectories** at checkpoints $\tau_0, \dots, \tau_L$, which are essential for behavior diversity. Here, τ_ℓ is the ℓ -th time checkpoint during the inner evaluation of heuristic h on instance j , and $A_{h,j}^{\tau_\ell}$ denotes the Pareto archive at that checkpoint. The final checkpoint τ_L is the end of the evaluation, so $A_{h,j}^{\tau_L}$ is the last archive state.

3 Quality Metrics

Hypervolume. With reference point r , the scaled hypervolume is:

$$\widetilde{HV}(P, r) = \frac{HV(P, r)}{\prod_{m=1}^M \max(|r_m|, 10^{-12})}, \quad HV_{ind}(h) = \frac{1}{J} \sum_{j=1}^J \widetilde{HV}(P_{h,j}, r).$$

Preference coverage. For preference vectors $\Lambda = \{\lambda^{(k)}\}$ (default for bi-objective: $(0.9, 0.1), (0.7, 0.3), (0.5, 0.5)$), normalize objectives:

$$\hat{y}_m = \max\left(0, \frac{y_m - z_m^{ideal}}{\max(z_m^{nadir} - z_m^{ideal}, \epsilon)}\right), \quad \epsilon = 10^{-12}.$$

The augmented scalarizing function is:

$$ASF(y \mid \lambda) = \max_m (\lambda_m \hat{y}_m) + \rho \sum_{m=1}^M \lambda_m \hat{y}_m, \quad \rho = 0.05.$$

The preference score of a front is $S(P, \lambda) = \min_{y \in P} ASF(y \mid \lambda)$, and the average preference-performance vector is $SP_h = \frac{1}{J} \sum_{j=1}^J [S(P_{h,j}, \lambda^{(1)}), \dots, S(P_{h,j}, \lambda^{(K)})]$. The coverage loss is:

$$CoverageLoss(h) = \text{mean}(SP_h) + \text{std}(SP_h).$$

Lower is better.

Population-level contribution. For population $\mathcal{P}$, the combined front on instance j is $P_{\mathcal{P},j} = ND(\bigcup_{h \in \mathcal{P}} P_{h,j})$. The hypervolume contribution of h is:

$$C_{HV}(h; \mathcal{P}) = \max\left(0, HV_{\mathcal{P}} - HV_{\mathcal{P} \setminus \{h\}}\right).$$

4 Pareto Behavior Trajectory (PBT)

At each checkpoint τ_ℓ , the archive is converted into a behavior vector. First, compute preference scores of the checkpoint archive:

$$BScore_{h,j}^{\tau_\ell, k} = S(A_{h,j}^{\tau_\ell}, \lambda^{(k)}), \quad k = 1, \dots, K.$$

If the checkpoint archive is empty or invalid, every $BScore$ is set to 10^6 and the normalized size is 0. Otherwise, the normalized archive size is:

$$Q_{h,j}^{\tau_\ell} = \frac{|A_{h,j}^{\tau_\ell}|}{\max_{\ell'=0, \dots, L} |A_{h,j}^{\tau_{\ell'}}|}.$$

The behavior vector at checkpoint τ_ℓ is:

$$b_{h,j}^{\tau_\ell} = [BScore_{h,j}^{\tau_\ell, 1}, \dots, BScore_{h,j}^{\tau_\ell, K}, Q_{h,j}^{\tau_\ell}] \in \mathbb{R}^{K+1}.$$

The **Pareto Behavior Trajectory** of heuristic h on instance j is the sequence of these vectors:

$$PBT_{h,j} = (b_{h,j}^{\tau_0}, b_{h,j}^{\tau_1}, \dots, b_{h,j}^{\tau_L}).$$

Across all J instances: $PBT_h = \{PBT_{h,1}, \dots, PBT_{h,J}\}$.

This captures **which Pareto regions are improved, when, and how fast**, as well as how the archive grows over time.

5 Behavior Distance with Dynamic Time Warping (DTW)

Two heuristics may follow similar improvement patterns at different speeds. DTW is used to compare their PBTs robustly.

Let $U = (u_1, \dots, u_n)$ and $V = (v_1, \dots, v_m)$ with $u_i, v_j \in \mathbb{R}^{K+1}$. The local cost is Euclidean distance:

$$c(i, j) = \|u_i - v_j\|_2 = \sqrt{\sum_{p=1}^{K+1} (u_{i,p} - v_{j,p})^2}.$$

DTW dynamic programming:

$$D(0, 0) = 0, \quad D(i, 0) = +\infty, \quad D(0, j) = +\infty,$$

$$D(i, j) = c(i, j) + \min\{D(i-1, j), D(i, j-1), D(i-1, j-1)\}.$$

The normalized DTW distance is:

$$d_{DTW}(U, V) = \frac{D(n, m)}{n + m}.$$

For two heuristics h_a, h_b , the behavior distance is averaged over the minimum number of evaluated instances:

$$d_{beh}(h_a, h_b) = \frac{1}{J'} \sum_{j=1}^{J'} d_{DTW}(PBT_{h_a, j}, PBT_{h_b, j}), \quad J' = \min(J_a, J_b).$$

This is a **measured difference of executed search dynamics**, not source-code similarity.

6 Behavior Diversity Calculation

Behavior diversity measures how different a heuristic’s search trajectory is from its nearest neighbors in the current population.

Step 1 — Compute distances. For heuristic h , compute its DTW distance to every other heuristic in the population:

$$\mathcal{D}_h = \{d_{beh}(h, g) \mid g \in \mathcal{P}_{-h}\}, \quad \mathcal{P}_{-h} = \mathcal{P} \setminus \{h\}.$$

Step 2 — Sort distances. Order the distances from smallest to largest:

$$d_{(1)} \leq d_{(2)} \leq \dots \leq d_{(N-1)}.$$

Step 3 — Average nearest neighbors. Let $k' = \min(k, |\mathcal{P}_{-h}|)$. The behavior diversity is:

$$D_{beh}(h) = \frac{1}{k'} \sum_{i=1}^{k'} d_{(i)}, \quad k = 3.$$

If there are no other heuristics, the default is $D_{beh}(h) = 1.0$.

What this means:

- A large $D_{beh}(h)$ means h searches the Pareto space in a way that is **far from every other heuristic** in the population.
- A small $D_{beh}(h)$ means at least k other heuristics follow a **similar improvement pattern**.
- Two heuristics with similar final hypervolume can have very different PBTs (e.g., one improves the (0.9, 0.1) region early, the other improves (0.1, 0.9) early). DTW captures this, and D_{beh} rewards both.

Why DTW instead of direct Euclidean?

A direct checkpoint-wise Euclidean distance would penalize two heuristics that follow the same improvement pattern at different speeds. DTW aligns steps flexibly, so it compares the **shape** of behavior rather than the exact timing.

7 Population Selection and Evolution

Selection. Heuristics are selected by non-dominated sorting on $(C_{HV}(h), D_{beh}(h))$. A heuristic survives if it either contributes strongly to population HV or has distinct behavior, or lies on the Pareto front of the two. If the front overflows, crowding distance in the (C_{HV}, D_{beh}) plane is used. Tie-breaking then prefers $(CD(h), HV_{ind}(h), -CoverageLoss(h))$ in descending order.

Parent selection. The weakest preference region is $k^* = \arg \max_k SP_{\mathcal{P}, k}$. Parents are selected with complementary roles:

- $p_1 = \arg \max_h HV_{ind}(h)$ (strongest individual);

- $p_2 = \arg \max_{h \neq p_1} C_{HV}(h)$ (strongest contributor);
- $p_3 = \arg \max_h D_{beh}(h)$ among useful heuristics not already selected (most distinct behavior), or alternatively $p_3 = \arg \min_h SP_{h,k^*}$ (best coverage of the weakest region).

LLM operators. New heuristics are generated by:

- **i1**: initial generation;
- **e1**: recombine multiple parents;
- **e2**: find a common backbone and generate a variant;
- **m1**: mutate selection or neighborhood logic;
- **m2**: mutate parameters or scoring formulas.

Optional clustering by a secondary LLM groups parents by semantic logic to reduce code-level redundancy, but behavior diversity is always computed **after execution** via PBT and DTW.

Full algorithm.

```

Input: LLM, evaluator with archive trajectories, population size N, sample budget T
Initialize: P = empty population; enable return_mo_trace in evaluator

while |P| < N:
    generate h with i1; evaluate h; compute fronts, HV, PBT; register h

while budget remains:
    find weakest preference region
    select parents by (HV_ind, C_HV, D_beh)
    optionally cluster parents by LLM
    generate h_new by e1/e2/m1/m2
    evaluate h_new; compute PBT and metrics; register h_new

when a full batch is ready:
    pool = old population + new batch
    compute C_HV and D_beh for all
    select by non-dominated sorting on (C_HV, D_beh)
    use crowding distance if needed

Output: final heuristic population, PBC report, checkpoints

```

8 Full Algorithm

8.1 Input and Initialization

Input:

- **LLM**: the heuristic-generation language model
- **evaluator**: a multi-objective evaluator that returns Pareto archive trajectories
- **N**: population size
- **T**: maximum sample budget
- **\Lambda**: set of preference vectors
- **r**: reference point for hypervolume

- k : number of nearest neighbors for behavior diversity (default 3)

Configuration:

- Enable `return_mo_trace` in the evaluator so that checkpoints $\tau_0, \dots, \tau_L$ and their archives $A_{h,j}^{\tau_\ell}$ are returned.

Initialize:

- $\mathcal{P} \leftarrow \emptyset$ (population of heuristic programs)
- $t \leftarrow 0$ (sample counter)

8.2 Initial Population Construction

```
while |P| < N and t < T:
  prompt <- build_i1_prompt()
  h <- LLM.generate(prompt)
  t <- t + 1

  for each instance j = 1 .. J:
    result_j <- evaluator.evaluate(h, instance_j)
    P_{h,j} <- result_j["front"]
    A_{h,j}^{\tau_0..L} <- result_j["trajectory"]
  end for

  compute HV_ind(h)
  compute CoverageLoss(h)
  compute PBT_h = {PBT_{h,1}, ..., PBT_{h,J}}

  register h in P with all metrics
end while
```

8.3 Evolution Loop

```
while t < T:
  --- Identify weakest preference region ---
  SP_P <- (1/J) * sum_j [S(P_{P,j}, lambda^{\{1\}}), ..., S(P_{P,j}, lambda^{\{K\}})]
  k* <- argmax_k SP_P[k]

  --- Parent selection ---
  p_1 <- argmax_{h in P} HV_ind(h) (best individual)
  p_2 <- argmax_{h in P, h != p_1} C_HV(h) (best contributor)

  candidates <- {h in P | C_HV(h) > 0, h not in {p_1, p_2}}
  if candidates is not empty:
    p_3 <- argmax_{h in candidates} D_beh(h) (most distinct behavior)
  else:
    p_3 <- argmin_{h in P} SP_{h,k*} (best weak-region coverage)
  end if

  --- Optional clustering ---
  if cluster_LLM is available and |parents| >= 2:
    clusters <- cluster_LLM.group_by_logic(parents)
    selected_parents <- pick one from each cluster (prioritize p_1, p_2, p_3)
```

```

else:
    selected_parents <- [p_1, p_2, p_3]
end if

--- LLM operator selection ---
if |selected_parents| >= 2:
    op <- choose_uniformly({e1, e2})
else:
    op <- choose_uniformly({m1, m2})
end if

prompt <- build_prompt(op, selected_parents)
h_new <- LLM.generate(prompt)
t <- t + 1

--- Evaluate new heuristic ---
for each instance j = 1 .. J:
    result_j <- evaluator.evaluate(h_new, instance_j)
    P_{h_new,j} <- result_j["front"]
    A_{h_new,j}^{\tau_0..L} <- result_j["trajectory"]
end for

compute HV_ind(h_new), CoverageLoss(h_new)
compute PBT_{h_new} and register h_new in a staging batch B

--- Population update (triggered when |B| == batch_size or t == T) ---
if |B| >= batch_size or t == T:
    pool <- P union B

    for each h in pool:
        compute P_{pool,j} = ND(union_{g in pool} P_{g,j}) for each j
        compute HV_{pool} = (1/J) sum_j HV_tilde(P_{pool,j}, r)
        compute HV_{pool \ {h}} by removing h's points
        C_HV(h) <- max(0, HV_{pool} - HV_{pool \ {h}})
    end for

    for each h in pool:
        compute D_beh(h) = (1/k') sum_{i=1}^{k'} d_{(i)}
        where d_{(i)} are the sorted distances d_beh(h, g) for g in pool \ {h}
        and k' = min(k, |pool| - 1)
    end for

    P <- non_dominated_sort_select(pool, objective=(C_HV, D_beh), target_size=N)

    if |P| > N:
        apply crowding_distance_tiebreak(P, objectives=(C_HV, D_beh))
        keep best N by (CD, HV_ind, -CoverageLoss)
    end if

    B <- empty
end if
end while

```

8.4 Selection Subroutine

```

function non_dominated_sort_select(pool, objective=(C_HV, D_beh), target_size):
  fronts <- fast_non_dominated_sort(pool, objective)
  P_new <- empty

  for front in fronts (in rank order):
    if |P_new| + |front| <= target_size:
      P_new <- P_new union front
    else:
      cd <- compute_crowding_distance(front, objective)
      sort front by cd descending
      P_new <- P_new union first (target_size - |P_new|) from front
      break
    end if
  end for

  return P_new
end function

```

8.5 Output

- **Final population $\mathcal{P}$:** the set of N selected heuristic programs
- **PBC report:** population hypervolume, coverage loss, average behavior diversity, and per-heuristic metrics
- **Population checkpoints:** full population state and PBTs at each update epoch

9 Summary

The core selection objective is:

$$\max_h (C_{HV}(h), D_{beh}(h)),$$

where:

$$C_{HV}(h) = HV_{\mathcal{P}} - HV_{\mathcal{P} \setminus \{h\}},$$

and:

$$D_{beh}(h) = \frac{1}{k'} \sum_{i=1}^{k'} d_{(i)}, \quad k' = \min(3, |\mathcal{P}_{-h}|).$$

Each $d_{(i)}$ is a nearest-neighbor DTW distance between PBTs:

$$d_{beh}(h_a, h_b) = \frac{1}{J'} \sum_{j=1}^{J'} d_{DTW}(PBT_{h_a, j}, PBT_{h_b, j}), \quad J' = \min(J_a, J_b).$$

The PBT is:

$$PBT_{h, j} = (b_{h, j}^{\tau_0}, \dots, b_{h, j}^{\tau_L}),$$

with:

$$b_{h, j}^{\tau_\ell} = [S(A_{h, j}^{\tau_\ell}, \lambda^{(1)}), \dots, S(A_{h, j}^{\tau_\ell}, \lambda^{(K)}), Q_{h, j}^{\tau_\ell}].$$

Therefore, **behavior diversity in PBCLLM is the nearest-neighbor diversity of executed Pareto search trajectories**. It measures how differently a heuristic improves and expands Pareto archives over time, not how different its source code looks.